Δομές Δεδομένων

1η Εργασία

Μέρος Α

Υλοποιήσαμε την δοσμένη διεπαφή StringDoubleEndedQueue με χρήση generics, ώστε να μπορούμε να χειριζόμαστε ουρές με οποιονδήποτε τύπο δεδομένων, έτσι τροποποιήσαμε κατάλληλα τον ορισμό της διεπαφής.

Αποτιμώντας την πολυπλοκότητα των μεθόδων addFirst(), removeFirst(), addLast(), removeLast(), getFirst(), getLast(), παρατηρούμε ότι είναι O(1), που σημαίνει ότι ο χρόνος εκτέλεσης αυτών των μεθόδων είναι σταθερός και δεν εξαρτάται από το πλήθος των στοιχείων που υπάρχουν ήδη στην λίστα.

Αυτό συμβαίνει γιατί η λίστα διατηρεί και ενημερώνει πάντα έναν δείκτη που δείχνει στον πρώτο κόμβο της (head) και έναν δείκτη που δείχνει στον τελευταίο κόμβο της (tail). Έτσι εφόσον οι εξαγωγές και οι εισαγωγές κόμβων γίνονται μόνο από τα άκρα της λίστας, η πρόσβαση σε αυτά είναι άμεση χωρίς να χρειαστεί να διασχίσουμε τη λίστα.

Η μέθοδος size έχει επίσης πολυπλοκότητα O(1) επειδή κατά την υλοποίηση της διπλά συνδεδεμένης λίστας διατηρούμε έναν μετρητή (size), ο οποίος ενημερώνεται αναλόγως κατά την προσθήκη ή αφαίρεση ενός κόμβου.

Συγκεκριμένα: κατά την προσθήκη ενός νέου στοιχείου (μέσω addFirst() ή addLast()), ο μετρητής size αυξάνεται κατά 1. Κατά την αφαίρεση ενός στοιχείου (μέσω removeFirst() ή removeLast()), ο μετρητής size μειώνεται κατά 1. Η ενημέρωση του μετρητή size είναι μια απλή λειτουργία και δεν εξαρτάται από τον αριθμό των στοιχείων που υπάρχουν στη λίστα.

Μέρος Β

Στο Μέρος Β, χρησιμοποιήσαμε την υλοποίηση της ουράς από το Μέρος Α για να μετατρέψουμε μια προθεματική παράσταση σε ενθεματική, ως εξής:

Διαβάζουμε την είσοδο σε ένα String, με όνομα prefixExpression, χρησιμοποιώντας την βοηθητική κλάση Scanner.

Στην συνέχεια χρησιμοποιούμε την ουρά queue, που είναι μια StringDoubleEndedQueue όπως την έχουμε υλοποιήσει στο Μέρος Α και ακολουθούμε τα παρακάτω βήματα:

1. Διατρέχουμε το String prefixExpression χαρακτήρα προς χαρακτήρα από τον τελευταίο του χαρακτήρα προς τον πρώτο.
2. Αν ο χαρακτήρας είναι ψηφίο (τελεστέος), τότε χρησιμοποιώντας την μέθοδο addFirst() τον τοποθετούμε ως πρώτο στην ουρά queue.

3. Αν ο χαρακτήρας είναι τελεστής (+, -, *, /), τότε χρησιμοποιώντας την μέθοδο removeFirst() δύο φορές εξάγουμε τα δύο πρώτα στοιχεία της ουράς (operant1, operant2). Στη συνέχεια δημιουργούμε ένα νέο String infix το οποίο συνενώνει τους δύο τελεστέους με το τελεστή ως εξής:
   infix = "(" + operand1 + τελεστής + operand2 + ")".
4. Στην συνέχεια τοποθετούμε αυτό το String στην ουρά με την μέθοδο addFirst().
5. Επαναλαμβάνουμε τα παραπάνω βήματα για κάθε χαρακτήρα του String prefixExpression. Τελικά (αν η prefixExpression είναι έγκυρη) η ουρά θα περιέχει την ενθεματική παράσταση ως μοναδικό στοιχείο.

Παράδειγμα:

Έστω ότι η prefixExpression είναι: ***-1/23-/145**

Τότε σύμφωνα με τα παραπάνω βήματα θα έχουμε:

prefixExpression	Βήμα			queue
*-1/23-/145	2			5
*-1/23-/14	2		4	5
*-1/23-/1	2	1	4	5
*-1/23-/	3, 4		(1/4)	5
*-1/23-	3, 4			((1/4)-5)
*-1/23	2		3	((1/4)-5)
*-1/2	2	2	3	((1/4)-5)
*-1/	3, 4		(2/3)	((1/4)-5)
*-1	2	1	(2/3)	((1/4)-5)
*-	3 ,4		(1-(2/3))	((1/4)-5)
*	3, 4			((1-(2/3))*((1/4)-5))

Έτσι τελικά η ενθεματική παράσταση infixExpression είναι: **((1-(2/3))*((1/4)-5))**

Κατά την υλοποίηση αντιμετωπίστηκαν και σφάλματα που μπορούν να συμβούν αν η προθεματική παράσταση δεν είναι σε έγκυρη μορφή. Οι περιπτώσεις αυτές είναι:

1. Αν περιέχει και άλλα σύμβολα εκτός ψηφίων και τελεστών.
2. Αν ξεκινάει με ψηφίο.
3. Αν τελειώνει με τελεστή.
4. Αν περιέχει περισσότερους τελεστέους από όσους χρειάζονται.
5. Αν περιέχει περισσότερους τελεστές από όσους χρειάζονται.

Μέρος Γ

Στο Μέρος Γ, χρησιμοποιήσαμε την υλοποίηση της ουράς από το Μέρος Α για να ελέγξουμε αν μια ακολουθία DNA είναι Watson-Crick complemented palindrome (συμπληρωματικά παλίνδρομη), ως εξής:.

Αφού διαβάσουμε την ακολουθία DNA στο String dnaSequence, αρχικά με την μέθοδο isValidSequence(), εξετάζουμε αν η ακολουθία είναι έγκυρη, δηλαδή αν περιέχει μόνο τα γράμματα A, T, C, G ή είναι κενή.

Στην περίπτωση που είναι έγκυρη, με την μέθοδο isWCPalindrome() εξετάζεται αν είναι Watson-Crick complemented palindrome, ως εξής:

Αρχικά ελέγχουμε το μήκος της ακολουθίας, αν αυτό είναι περιττό τότε αποκλείεται να είναι Watson-Crick complemented palindrome. Αν όμως το μήκος είναι άρτιο, τότε αντιγράφουμε τα περιεχόμενά της στη δομή queue που είναι τύπου StringDoubleEndedQueue ουράς με δύο άκρα που κατασκευάσαμε στο Μέρος Α.

Στη συνέχεια αφαιρούμε επαναληπτικά το πρώτο στοιχείο της ουράς και το τελευταίο και ελέγχουμε αν είναι συμπληρωματικά (A-T, C-G). Όταν βρούμε ένα μη συμπληρωματικό ζευγάρι σταματάμε και η ακολουθία δεν είναι συμπληρωματική, αλλιώς συνεχίζουμε μέχρι να αφαιρεθούν όλα τα στοιχεία από την ουρά. Στην περίπτωση που αφαιρεθούν όλα, η ακολουθία είναι Watson-Crick complemented palindrome.

Επίσης η κενή ακολουθία είναι Watson-Crick complemented palindrome.

Το πρόγραμμα αυτό έχει επανάληψη που εξαρτάται από το πλήθος των χαρακτήρων της ακολουθίας οπότε εκτελείται σε χρόνο O(N). Επιπλέον, μετά το διάβασμα της εισόδου, η επεξεργασία της ουράς γίνεται με 1 loop.